
Data Structures

BS (CS) _Fall_2025

Lab_09 Task



Learning Objectives:

1. Binary Trees

Task 1: Student Record Management Using Binary Tree

A University Tracking System maintains student records in a Binary Tree.

Each node stores a student's:

- ID (integer, unique)
- Name (string)
- GPA (float)
- Semester (integer)

Key Details

- In a Binary Tree insertion occurs from left to right, make sure to retain that property in your code.
- All operations must be implemented recursively.
- No STL containers (like vector, queue, or stack) should be used

The system must support:

- Insert Student Record
- Display All Records (Inorder Traversal)
- Update Student Information
- Delete Student Record (remove node completely)
- Show Tree Properties:
 - Total Students (node count)
 - Leaf Nodes
 - Tree Height
- Find Top Student (Highest GPA)
- Find Average GPA
- Count Students Below a Given GPA

All operations must be recursive, without using STL containers.

Test Cases:

```
// ----- BASIC TESTS -----  
  
// Test 1: Insert a single student  
TEST(StudentTreeTest, InsertSingleStudent) {  
    StudentTree tree;  
    tree.insertStudent(1, "Ali", 3.2, 2);  
    EXPECT_EQ(tree.totalNodes(), 1);  
}
```

```

// Test 2: Insert multiple students
TEST(StudentTreeTest, InsertMultipleStudents) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.2, 2);
    tree.insertStudent(2, "Sara", 3.8, 4);
    tree.insertStudent(3, "Hira", 3.6, 3);
    tree.insertStudent(4, "Zain", 3.4, 1);
    tree.insertStudent(5, "Usman", 2.9, 2);
    EXPECT_EQ(tree.totalNodes(), 5);
}

// Test 3: Update student GPA
TEST(StudentTreeTest, UpdateStudentGPA) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.2, 2);
    tree.updateStudent(1, "Ali", 3.9, 2);
    EXPECT_EQ(tree.totalNodes(), 1);
}

// Test 4: Update non-existing student
TEST(StudentTreeTest, UpdateNonExistingStudent) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.2, 2);
    tree.updateStudent(99, "None", 2.0, 1);
    EXPECT_EQ(tree.totalNodes(), 1);
}

// Test 5: Delete student (middle of tree)
TEST(StudentTreeTest, DeleteStudent) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.2, 2);
    tree.insertStudent(2, "Sara", 3.8, 4);
    tree.insertStudent(3, "Hira", 3.6, 3);
    tree.deleteStudent(2);
    EXPECT_GE(tree.totalNodes(), 2);
}

// Test 6: Delete non-existing student
TEST(StudentTreeTest, DeleteNonExistingStudent) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.2, 2);
    tree.deleteStudent(10);
    EXPECT_EQ(tree.totalNodes(), 1);
}

```

```
// ----- PROPERTY TESTS -----
```

```
TEST(StudentTreeTest, CountTotalNodes) {  
    StudentTree tree;  
    tree.insertStudent(1, "A", 3.0, 1);  
    tree.insertStudent(2, "B", 3.1, 2);  
    tree.insertStudent(3, "C", 3.2, 3);  
    tree.insertStudent(4, "D", 3.4, 4);  
    tree.insertStudent(5, "E", 3.5, 5);  
    EXPECT_EQ(tree.totalNodes(), 5);  
}
```

```
TEST(StudentTreeTest, CountLeafNodes) {  
    StudentTree tree;  
    tree.insertStudent(1, "A", 3.0, 1);  
    tree.insertStudent(2, "B", 3.1, 2);  
    tree.insertStudent(3, "C", 3.2, 3);  
    EXPECT_GE(tree.totalLeaves(), 1);  
}
```

```
TEST(StudentTreeTest, HeightOfTree) {  
    StudentTree tree;  
    tree.insertStudent(1, "Ali", 3.1, 1);  
    tree.insertStudent(2, "Sara", 3.2, 2);  
    tree.insertStudent(3, "Zain", 3.3, 3);  
    tree.insertStudent(4, "Hira", 3.4, 4);  
    EXPECT_GE(tree.heightTree(), 1);  
}
```

```
// ----- GPA TESTS -----
```

```
TEST(StudentTreeTest, AverageGPAWithMultipleStudents) {  
    StudentTree tree;  
    tree.insertStudent(1, "Ali", 3.0, 1);  
    tree.insertStudent(2, "Sara", 3.5, 2);  
    tree.insertStudent(3, "Hira", 4.0, 3);  
    tree.insertStudent(4, "Usman", 2.5, 4);  
    tree.insertStudent(5, "Zain", 3.3, 2);  
    tree.averageGPA();  
    EXPECT_EQ(tree.totalNodes(), 5);  
}
```

```
TEST(StudentTreeTest, TopStudentIdentification) {  
    StudentTree tree;
```

```

    tree.insertStudent(1, "Ali", 3.1, 1);
    tree.insertStudent(2, "Sara", 3.8, 2);
    tree.insertStudent(3, "Hira", 3.9, 3);
    tree.insertStudent(4, "Zain", 3.2, 4);
    tree.topStudent(); // should print Hira
    EXPECT_GE(tree.totalNodes(), 4);
}

// ----- EDGE CASE TESTS -----

TEST(StudentTreeTest, EmptyTreeNodes) {
    StudentTree tree;
    EXPECT_EQ(tree.totalNodes(), 0);
}

TEST(StudentTreeTest, EmptyTreeHeight) {
    StudentTree tree;
    EXPECT_EQ(tree.heightTree(), -1);
}

TEST(StudentTreeTest, DeleteEmptyTree) {
    StudentTree tree;
    tree.deleteStudent(10);
    EXPECT_EQ(tree.totalNodes(), 0);
}

TEST(StudentTreeTest, InsertThenDeleteSingleNode) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.1, 2);
    tree.deleteStudent(1);
    EXPECT_LE(tree.totalNodes(), 1);
}

TEST(StudentTreeTest, MultipleInsertAndPartialDelete) {
    StudentTree tree;
    tree.insertStudent(1, "A", 3.0, 1);
    tree.insertStudent(2, "B", 3.1, 1);
    tree.insertStudent(3, "C", 3.2, 1);
    tree.insertStudent(4, "D", 3.3, 2);
    tree.insertStudent(5, "E", 2.9, 3);
    tree.insertStudent(6, "F", 3.5, 4);
    tree.deleteStudent(3);
    tree.deleteStudent(5);
    EXPECT_GE(tree.totalNodes(), 3);
}

```

```

TEST(StudentTreeTest, ReinsertAfterDelete) {
    StudentTree tree;
    tree.insertStudent(1, "Ali", 3.0, 1);
    tree.deleteStudent(1);
    tree.insertStudent(2, "Sara", 3.5, 2);
    tree.insertStudent(3, "Hira", 3.6, 3);
    EXPECT_EQ(tree.totalNodes(), 2);
}

TEST(StudentTreeTest, DeepTreeProperties) {
    StudentTree tree;
    // create a deeper tree (7 nodes)
    tree.insertStudent(1, "Ali", 3.0, 1);
    tree.insertStudent(2, "Sara", 3.2, 2);
    tree.insertStudent(3, "Hira", 3.4, 3);
    tree.insertStudent(4, "Zain", 3.1, 4);
    tree.insertStudent(5, "Usman", 3.7, 5);
    tree.insertStudent(6, "Taha", 3.5, 6);
    tree.insertStudent(7, "Hamza", 2.9, 7);

    EXPECT_GE(tree.totalNodes(), 7);
    EXPECT_GE(tree.heightTree(), 2);
    EXPECT_GE(tree.totalLeaves(), 2);
}

```

Task 2: Student Record Management Using Binary Tree

Index StudentID Name GPA Semester

0	101	Ali	3.2	2
1	102	Sara	3.6	4
2	103	Hira	3.4	3
3	104	Bilal	2.9	1
4	105	Zain	3.7	5
5	106	Ayesha	3.8	4
6	107	Farhan	3.1	2
7	108	Sana	3.5	6
8	109	Rida	2.7	1
9	110	Imran	3.9	8

Part 1 Draw the Binary Tree:

The array represents the tree in level order (top to bottom, left to right).

Draw the tree manually on your paper using these relationships.

It should have 4 levels when complete.

Part 2 Find and Write the Following Properties

- Root node:
(Who is at the top of the tree?)
- Parent of “Zain”:
(Find using the index relationship.)
- Children of “Sara”:
(Left and right nodes connected to Sara.)
- Siblings of “Ayesha”:
(Node with the same parent as Ayesha.)
- Total number of nodes:
(Count all students in the tree.)
- Number of leaf nodes:
(Nodes that have no children.)
- Height of the tree:
(Number of edges from root to deepest leaf.)
- Left subtree of “Ali”: